

IEEE BUBT STUDENT BRANCH
IoTrix – Hackathon 1.0

PROJECT REPORT

Smart Multi-Sensor IoT Monitoring & Control System

ESP32-Based Local Web Server with Firebase Realtime Cloud Integration

Team Members	Alifa ara maisha, Md saif siam, Nahian Piyal, Sajjadul Islam, Sumaiya Akhter mim, Tanziba Zaman elmi, Somaiya akhter
Event	IoT Workshop & Hackathon — IoTrix 1.0
Organized By	IEEE BUBT Student Branch
Microcontroller	ESP32 DevKit V1 (Wrover Module)
Report Date	August 7, 2026

1. Abstract

This report documents the design, development, and verification of a multi-sensor Internet of Things (IoT) monitoring and control system, undertaken as part of the IoT Workshop and IoTrix Hackathon 1.0, organized by the IEEE BUBT Student Branch. The project uses an ESP32 microcontroller as the central processing and connectivity unit, interfacing with a range of sensors including an RFID-RC522 reader, a DHT11 temperature and humidity sensor, an HC-SR04 ultrasonic distance sensor, flame/IR detection modules, a capacitive soil moisture sensor, and a relay module for actuation of external loads. The work covers two complementary operating modes: (1) a local web server hosted directly on the ESP32 for serving a control interface to devices on the same network, and (2) a cloud-connected mode using Google Firebase Realtime Database for global access, real-time synchronization, and remote control. The report presents the objectives, tools, circuit design, firmware implementation, testing procedure, results, and conclusions of the work carried out.

2. Introduction

The Internet of Things (IoT) refers to the technology that allows everyday objects to connect to the internet and communicate with each other, collecting, transmitting, and acting on data to make devices smarter and more efficient. A typical IoT system works in four stages: a device collects data using sensors, the collected data is sent over the internet, the data is processed (often in the cloud), and finally the device or a user takes action based on the processed information.

This project was carried out as part of a structured IoT workshop and hackathon that guided participants from the fundamentals of microcontrollers and sensors through to a fully networked, cloud-integrated embedded system. The exercise was designed to build practical competency in embedded firmware development, circuit design, sensor interfacing, local web server hosting, and cloud database integration using industry-relevant tools.

2.1 Objectives

- Understand the fundamentals of IoT architecture and the role of microcontrollers within it.
- Gain hands-on experience with the ESP32 development board and its GPIO/peripheral capabilities.
- Interface multiple sensor types (RFID, temperature/humidity, ultrasonic, flame/IR, soil moisture) with a single controller.
- Implement a relay-based actuation module for controlling external electrical loads safely.
- Develop and test firmware in the Arduino IDE using ESP32-specific libraries.
- Host a local web server directly on the ESP32 to serve a control interface on the local network.
- Integrate the system with Firebase Realtime Database for cloud-based monitoring and control.
- Verify hardware wiring, power stability, and network configuration through structured checklists.

3. Tools, Platforms, and Applications Used

The following software tools and platforms were used throughout the design, simulation, and development process:

Arduino IDE 2.3.10	Primary firmware development environment for writing, compiling, and uploading C/C++ code to the ESP32.
Wokwi	Online ESP32 simulator used for testing circuit logic and code behavior before physical assembly.
circuito.io	Online circuit design assistant used to validate component wiring and generate reference schematics.
Circuit Designer	All-in-one circuit design IDE used for laying out and documenting the breadboard circuit.
Firebase Realtime Database (RTDB)	Cloud-hosted NoSQL database used as the central hub for remote data storage and real-time synchronization.

4. Hardware Overview

4.1 Microcontroller — ESP32 DevKit V1

The ESP32 DevKit V1 (Wrover module) was selected as the core controller for this project. It integrates Wi-Fi and Bluetooth connectivity, a dual-core processor, and a large number of GPIO pins, making it well suited for simultaneously managing multiple sensors, a relay actuator, and network communication. Its low cost and strong community/library support (particularly for Firebase and web server functionality) made it the preferred choice over a standard Arduino Uno for this project's networked requirements.

4.2 Sensors and Modules Used

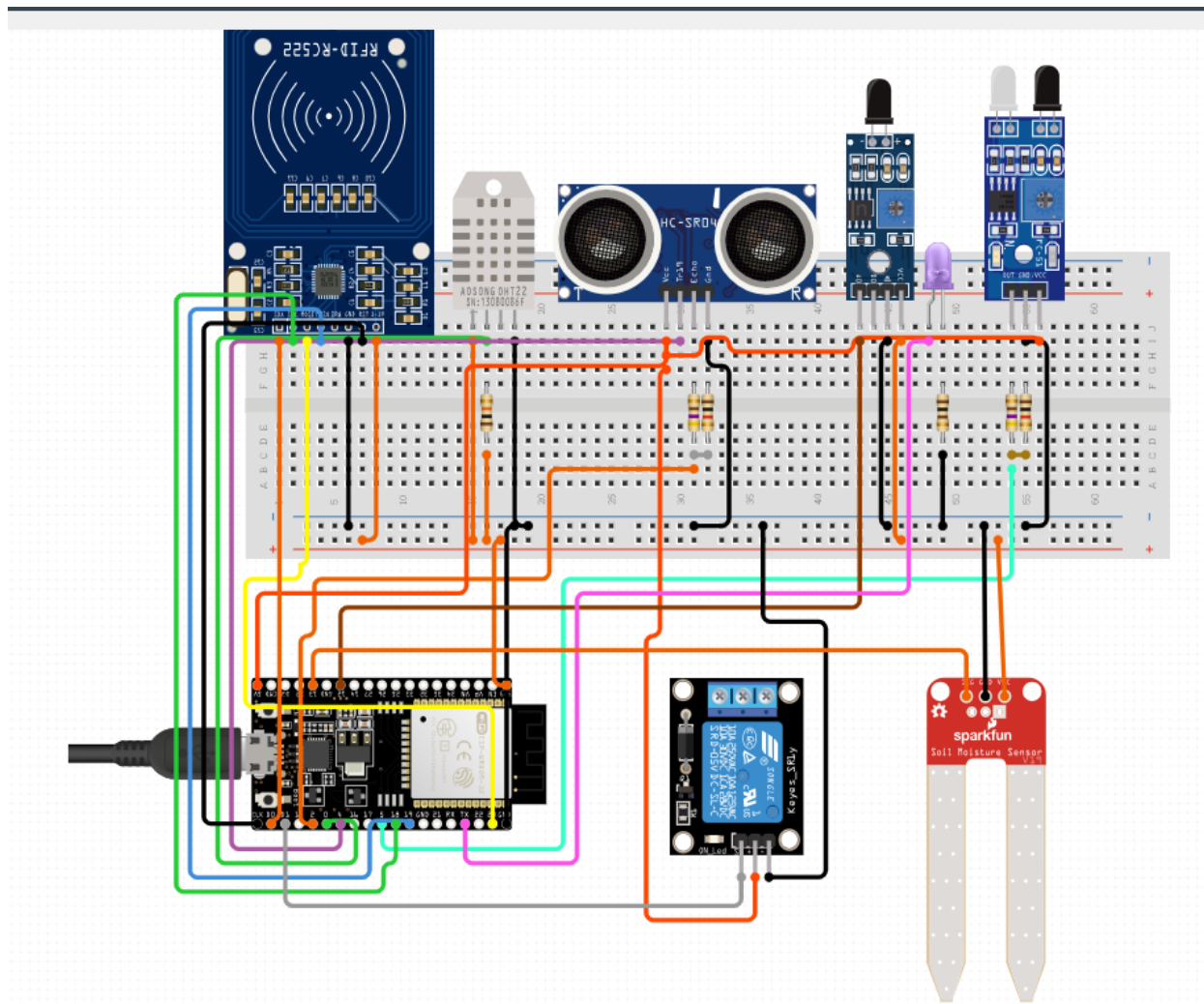
The system integrates the following sensors and modules, selected from the broader sensor family studied during the workshop:

RFID-RC522 Reader	13.56 MHz RFID module used for tag/card-based identification and access control.
DHT11 Sensor	Digital sensor used to measure ambient temperature and relative humidity.
HC-SR04 Ultrasonic Sensor	Distance-ranging sensor using ultrasonic pulses, used for proximity/distance measurement.
Flame / IR Sensor Modules (x2)	Infrared-based sensors used for flame and obstacle/heat-source detection.
Capacitive Soil Moisture Sensor	Analog sensor used to measure the moisture content of soil for environmental monitoring.
1-Channel Relay Module (5V)	Electromechanical switch used to control a higher-power external load (e.g., a pump or light) from a GPIO signal.

4.3 Circuit Design and Wiring

All modules were mounted on a dual-row breadboard and wired to the ESP32's GPIO, power, and ground rails, with pull-up/current-limiting resistors placed in line with the sensor signal paths where required. Power was

distributed via the breadboard's positive and negative rails, sourced from the ESP32's 3V3/5V pins and regulated as needed per module. The complete wiring layout, including all sensor-to-microcontroller and relay connections, is shown below.

Figure 1 — Complete Breadboard Wiring Diagram

RFID-RC522, DHT11, HC-SR04 ultrasonic sensor, flame/IR sensor modules, soil moisture sensor, and a relay module wired to the ESP32 DevKit.

4.4 Pin Mapping Summary

DHT11 Data	GPIO 4
Relay Control Signal	GPIO 2 (via current-limiting resistor)
Soil Moisture Sensor	Analog input (ADC pin)
HC-SR04 Trig / Echo	Dedicated digital output / input pins
RFID-RC522	SPI interface (SDA, SCK, MOSI, MISO, RST)
Power Rails	5 V / 3.3 V and GND distributed via breadboard rails

Note: exact GPIO assignments were mapped and verified against the ESP32 DevKit V1 pinout during the hardware verification stage described in Section 7.

5. Methodology

The project followed a two-part architecture, moving from a purely local implementation to a fully cloud-connected system. This staged approach allowed core sensor logic and web-serving functionality to be validated locally before introducing the additional complexity of internet-based synchronization.

5.1 Part 1 — Local Host & Web Server

In the first stage, the ESP32 was configured in Station (STA) mode and programmed to serve an HTML/CSS interface directly to any device connected to the same local network, without relying on an external cloud broker. This was implemented using the built-in `WebServer.h` library.

- A `WebServer` object was instantiated on port 80.
- URI handlers were registered (e.g. the root path `"/"`) to serve the HTML control page.
- `server.handleClient()` was called continuously within the main loop to process incoming client requests.

A simplified excerpt of the local web server firmware is shown below:

```
#include <WiFi.h>
#include <WebServer.h>

WebServer server(80);
String htmlPage = "<html><body> ... ";

void setup() {
  server.on("/", []() {
    server.send(200, "text/html", htmlPage);
  });
  server.begin();
}

void loop() {
  server.handleClient();
}
```

5.2 Part 2 — Firebase Cloud Integration

To overcome the range limitations of a purely local server, the system was extended to use Google Firebase Realtime Database (RTDB) as a central cloud hub. Firebase stores data as a hierarchical NoSQL/JSON tree (e.g. `/Home/LivingRoom/Temp`), enabling intuitive mapping of sensor and actuator states, and uses WebSockets to push updates instantly to all connected clients, avoiding the overhead of constant HTTP polling.

The cloud integration was implemented in three steps using the `Firebase_ESP_Client.h` library:

1. Configuration & Authentication — a `FirebaseConfig` object was set up with the project's API key and database URL, and an anonymous sign-in was used to authenticate the device.

2. Connection — `Firebase.begin()` was called with the configuration and authentication objects, and `Firebase.reconnectWiFi(true)` was enabled to maintain connectivity automatically.
3. Real-Time Data Streams — a persistent WebSocket stream was opened using `Firebase.RTDB.beginStream()` on a control path (e.g. `"/Controls"`), with a callback function used to react instantly to database changes, such as toggling the relay when a new value is written to `"/Relay1"`.

Excerpt — Firebase configuration and authentication:

```
#include <Firebase_ESP_Client.h>
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

void setup() {
  config.api_key = "YOUR_FIREBASE_API_KEY";
  config.database_url = "YOUR_DATABASE_URL";

  if (Firebase.signUp(&config, &auth, "", "")) {
    Serial.println("Auth Successful");
  }
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
}
```

Excerpt — real-time stream callback for relay control:

```
void streamCallback(FirebaseStream data) {
  if (data.dataPath() == "/Relay1") {
    digitalWrite(RELAY_PIN, data.stringData().toInt());
  }
}

void setupStream() {
  Firebase.RTDB.beginStream(&fbdo, "/Controls");
  Firebase.RTDB.setStreamCallback(&fbdo, streamCallback, nullptr);
}
```

6. Practical Tasks / Mini-Exercises

Alongside the main system build, four progressive hands-on tasks were completed to build and confirm core competency with individual sensors and actuators before integrating them into the full circuit shown in Figure 1. Each task uses a single LED as the output indicator, wired through a current-limiting resistor to a digital GPIO pin.

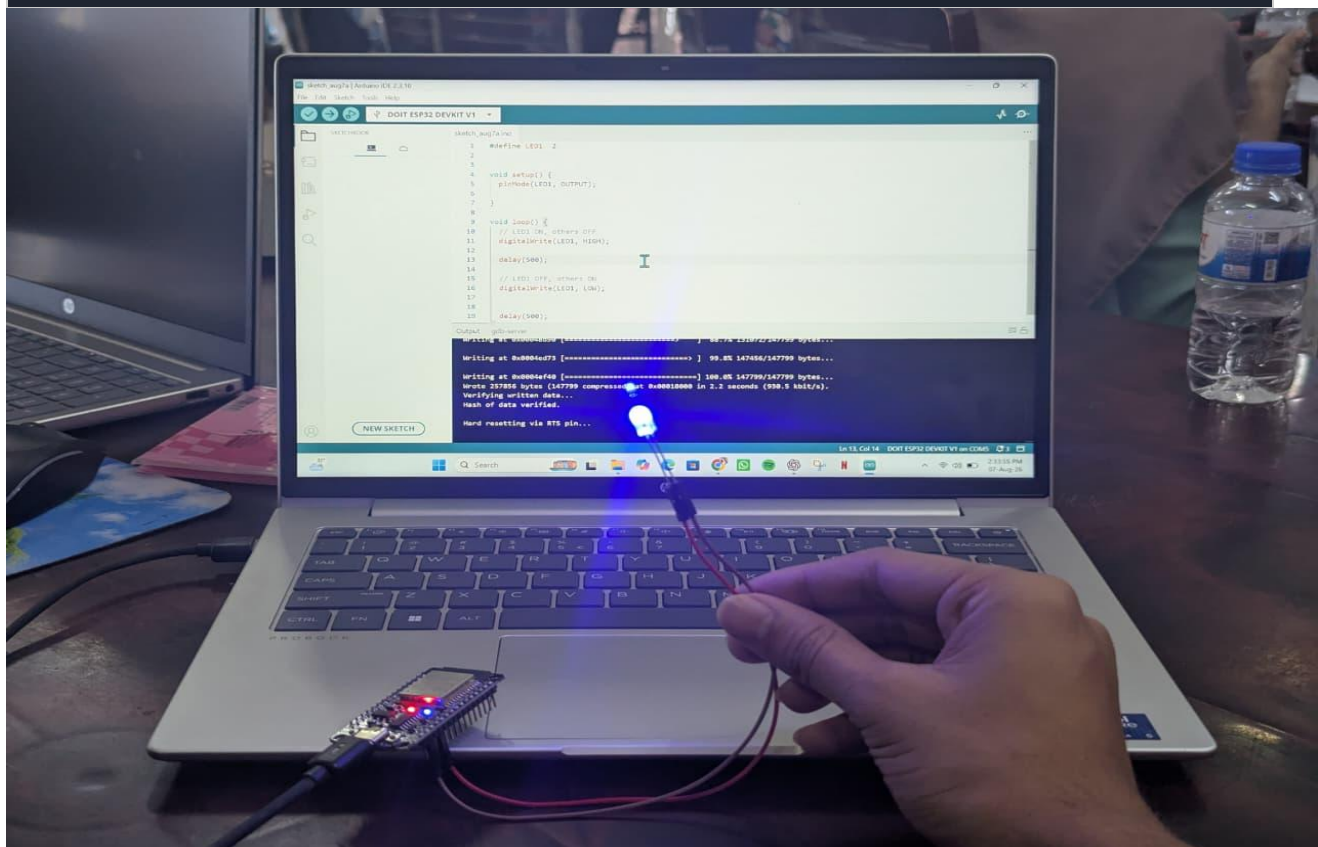
Task 1 — LED: Light Up and Blink

Objective: verify basic GPIO digital output control by turning an LED on and off at a fixed interval. This confirms the board, IDE toolchain, and wiring are functioning correctly before adding sensors.

```
#define LED_PIN 2

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // LED ON
  delay(1000);
  digitalWrite(LED_PIN, LOW);  // LED OFF
  delay(1000);
}
```



Task 2 — Ultrasonic Sensor with LED

Objective: use the HC-SR04 ultrasonic sensor to measure distance and light the LED whenever an object is detected within a defined threshold range, demonstrating sensor-driven digital output control.

```
#define TRIG_PIN 5
#define ECHO_PIN 18
#define LED_PIN 2
#define THRESHOLD_CM 15

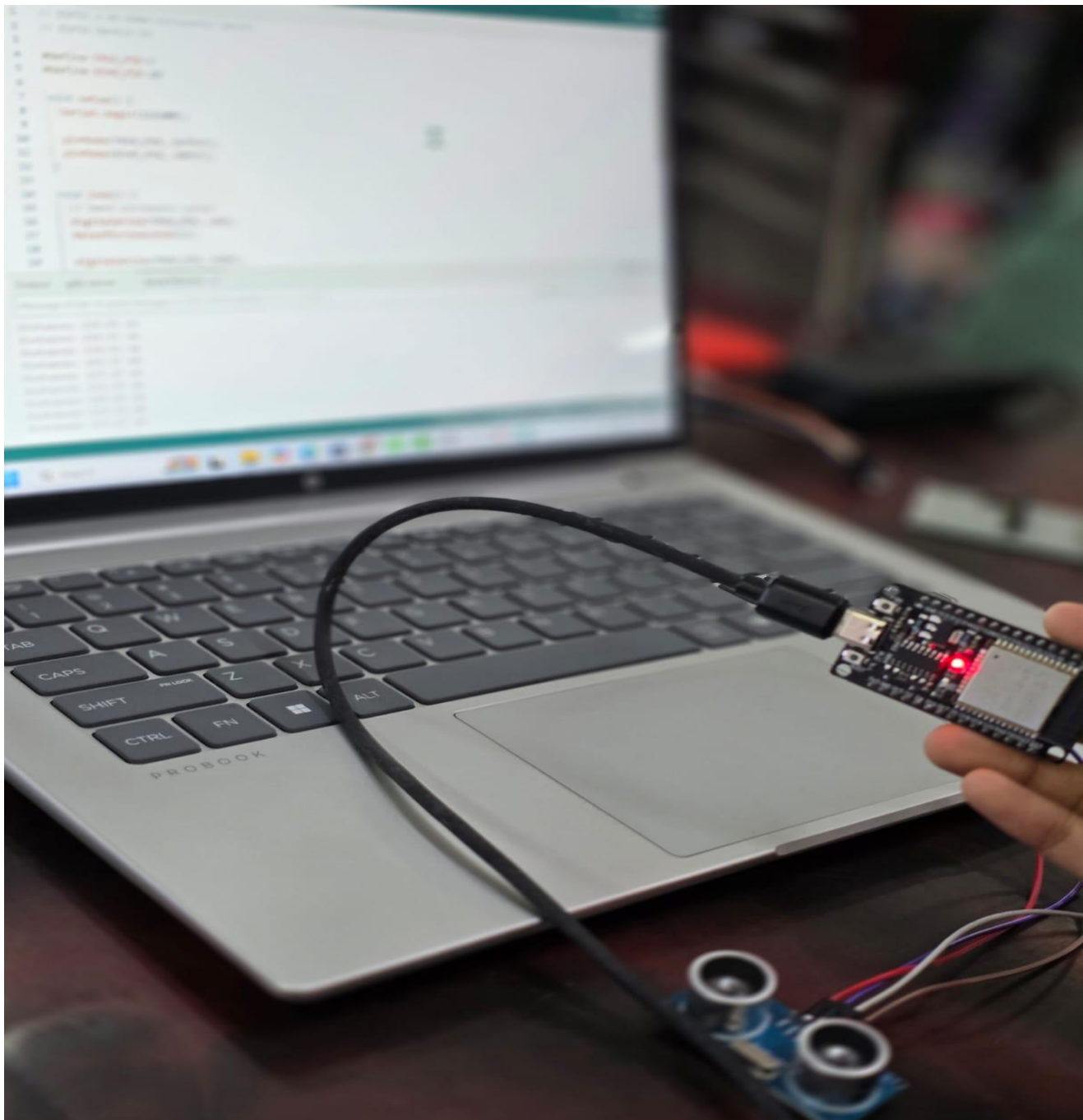
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
}

void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);
  float distanceCm = duration * 0.0343 / 2;

  Serial.print("Distance: "); Serial.println(distanceCm);

  if (distanceCm > 0 && distanceCm < THRESHOLD_CM) {
    digitalWrite(LED_PIN, HIGH); // Object detected nearby
  } else {
    digitalWrite(LED_PIN, LOW);
  }
  delay(300);
}
```



Task 3 — Soil Moisture Sensor with LED Blink

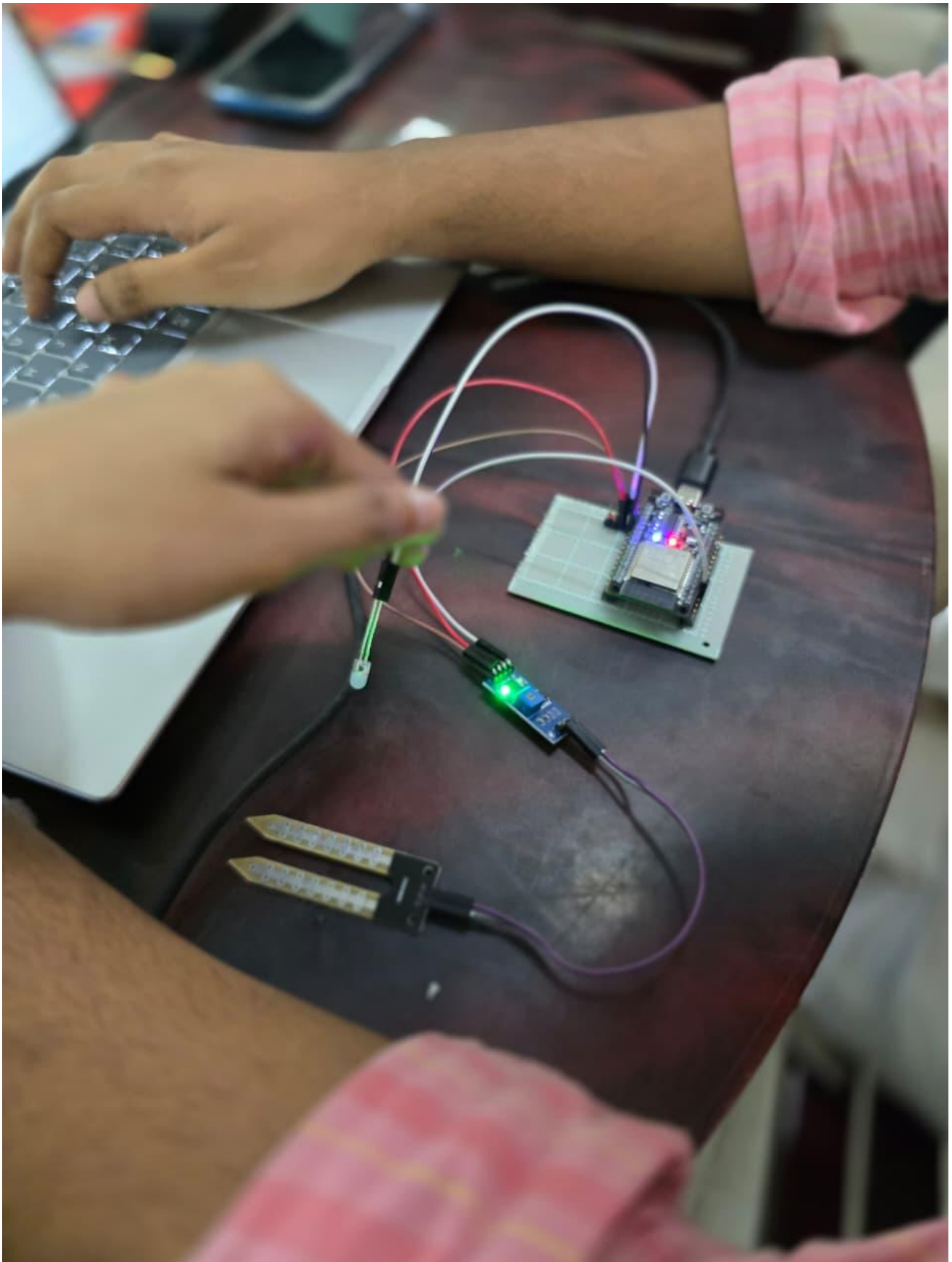
Objective: read the analog output of the capacitive soil moisture sensor and blink the LED at a rate/state that reflects soil dryness, indicating when the soil moisture falls below an acceptable level (e.g., for irrigation alerting).

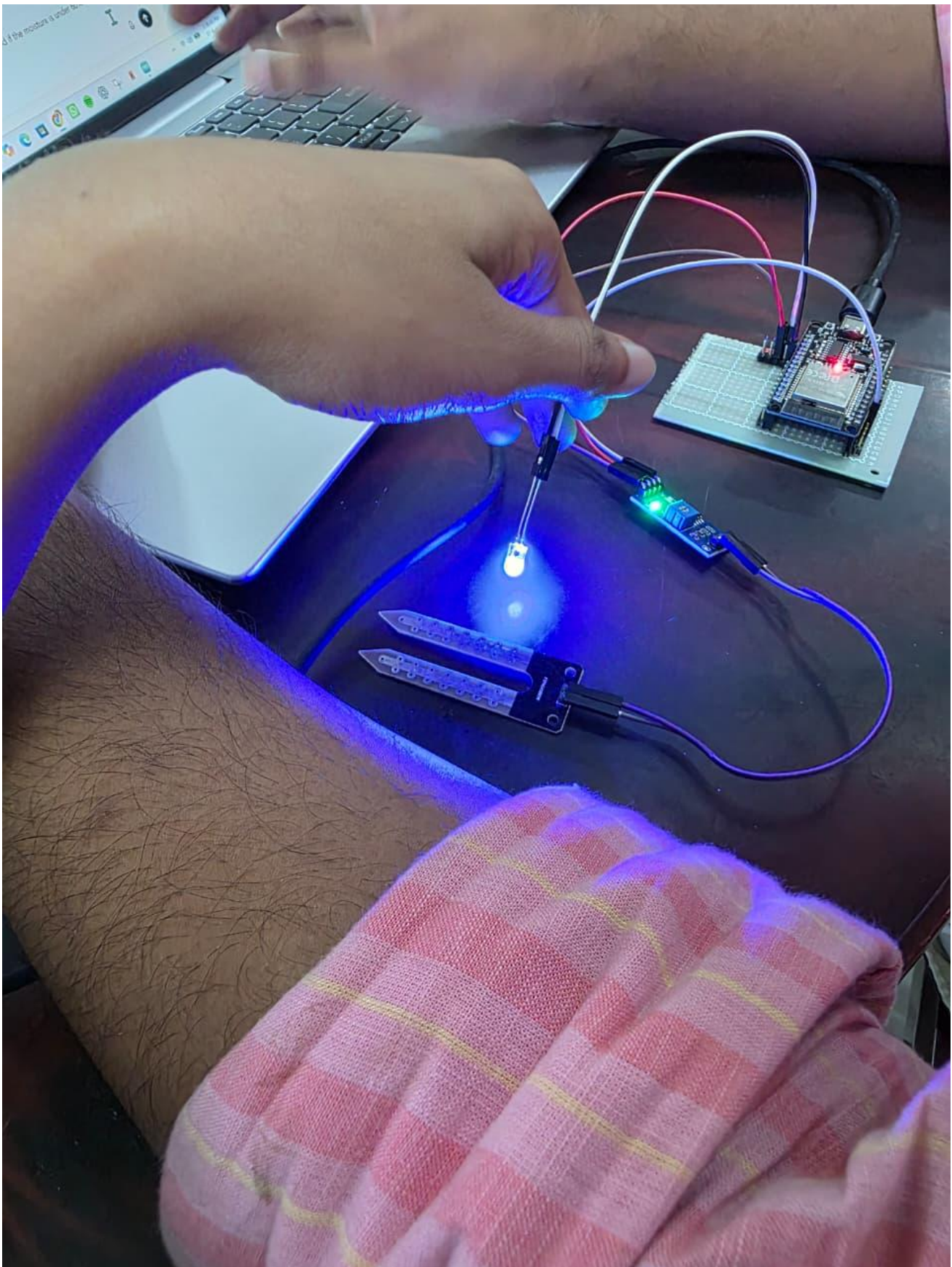
```
#define SOIL_PIN 34    // Analog (ADC) input
#define LED_PIN  2
#define DRY_THRESHOLD 2200    // Adjust based on calibration
```

```
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
}

void loop() {
  int soilValue = analogRead(SOIL_PIN);
  Serial.print("Soil Moisture Raw: "); Serial.println(soilValue);

  if (soilValue > DRY_THRESHOLD) {
    // Soil is dry -> blink LED as an alert
    digitalWrite(LED_PIN, HIGH);
    delay(200);
    digitalWrite(LED_PIN, LOW);
    delay(200);
  } else {
    // Soil is sufficiently moist -> LED off
    digitalWrite(LED_PIN, LOW);
  }
  delay(500);
}
```





Task 4 — Humidity and Temperature Sensor (DHT11) with LED

Objective: read ambient temperature and humidity from the DHT11 sensor using the DHT library, and switch the LED on whenever the temperature exceeds a defined comfort threshold, demonstrating environmental-condition-based actuation.

```
#include <DHT.h>

#define DHT_PIN    4
#define DHT_TYPE   DHT11
#define LED_PIN    2
#define TEMP_THRESHOLD_C 30.0

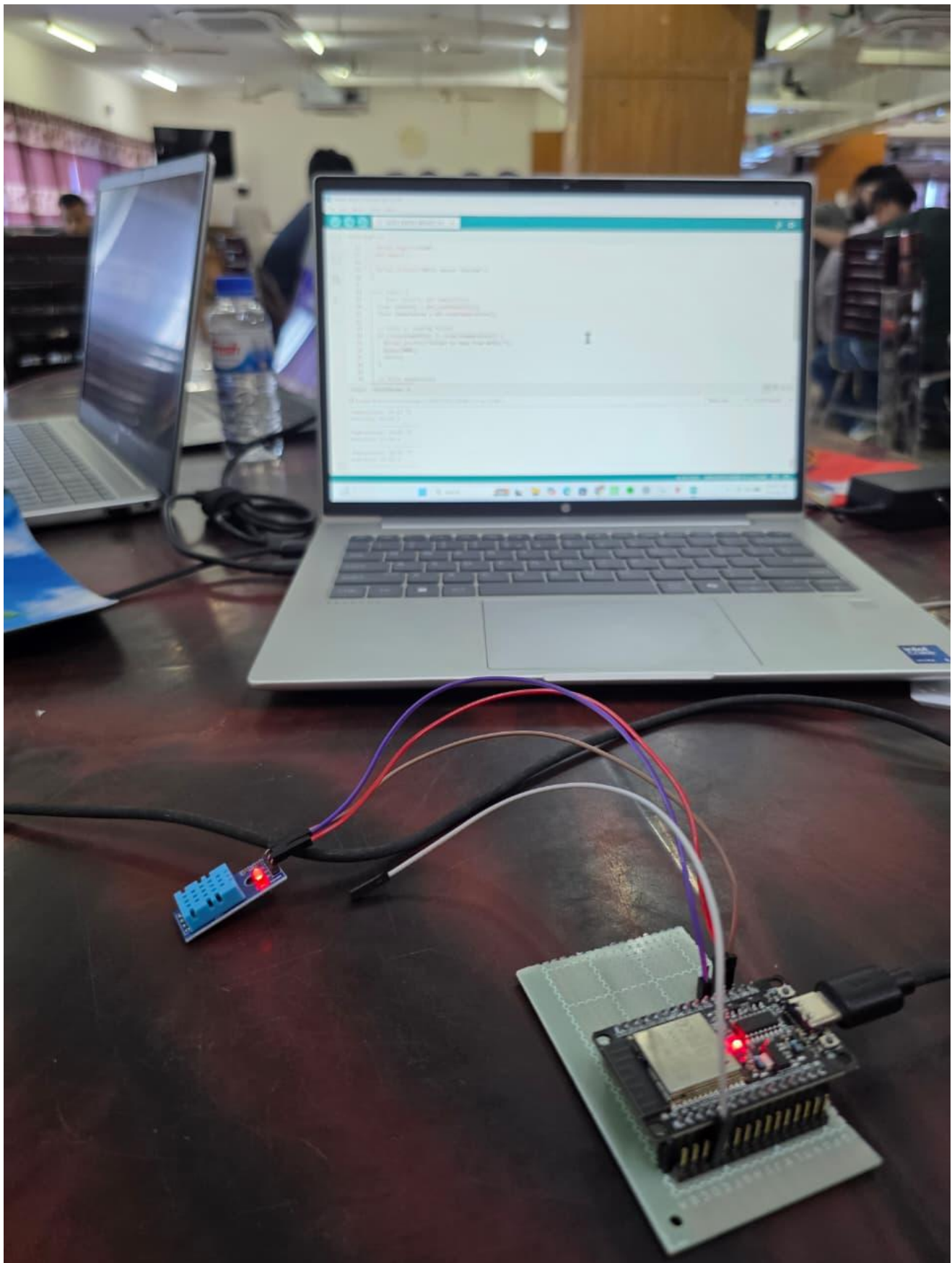
DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();
  float humidity    = dht.readHumidity();

  if (!isnan(temperature) && !isnan(humidity)) {
    Serial.print("Temp: "); Serial.print(temperature); Serial.print(" C, ");
    Serial.print("Humidity: "); Serial.print(humidity); Serial.println(" %");

    digitalWrite(LED_PIN, temperature > TEMP_THRESHOLD_C ? HIGH : LOW);
  } else {
    Serial.println("Failed to read from DHT sensor!");
  }
  delay(2000); // DHT11 requires >= 1s between reads
}
```



7. Hardware Verification and Testing

Before and after flashing firmware to the ESP32, structured verification checklists were used to confirm that the hardware was correctly and safely interfaced, for both the local-server and cloud-connected configurations.

7.1 Local Server Configuration — Circuit Verification

Connections Checklist

- ✓ ESP32 connected to PC via a high-quality data cable
- ✓ Sensor data pins correctly mapped (e.g., DHT11 on GPIO 4)
- ✓ LED/relay actuators wired via current-limiting resistors (e.g., GPIO 2)
- ✓ Continuity check passed — hardware correctly interfaced for the local web server environment

7.2 Cloud Server Configuration — Circuit Verification

Verification Checklist

- ✓ Wi-Fi internet breakout confirmed (WAN access available)
- ✓ Firewall settings checked to allow WebSocket ports
- ✓ Power stability verified — TLS encryption increases peak current draw
- ✓ Cloud circuit schematic reviewed and marked verified

8. Results

The completed system successfully demonstrated end-to-end functionality across both operating modes. On the local network, the ESP32 reliably served its control interface to connected clients with low latency, confirming correct implementation of the WebServer.h-based routing and client handling. Following the transition to Firebase, sensor readings and actuator states (in particular, relay switching) synchronized in real time between the physical hardware and the Firebase Realtime Database, confirming that the WebSocket-based streaming mechanism eliminated the need for constant polling while maintaining responsive control.

All sensors integrated in the circuit — the RFID-RC522 reader, DHT11 temperature/humidity sensor, HC-SR04 ultrasonic sensor, flame/IR modules, and soil moisture sensor — were verified individually against the wiring diagram in Figure 1, and the relay module was confirmed to switch correctly in response to both local and cloud-issued commands.

9. Challenges Encountered

- Correctly mapping multiple sensor signal lines to available ESP32 GPIO pins without conflicts, given the number of simultaneously connected modules.
- Ensuring stable power delivery across all sensors and the relay module from the ESP32's onboard regulator and breadboard rails.

- Managing the additional current draw introduced by TLS encryption once the system moved from local hosting to secure cloud (Firebase) communication.
- Configuring firewall and network settings to permit persistent WebSocket connections required for real-time Firebase streaming.

10. Conclusion

This project provided practical, end-to-end experience in building a networked embedded system — from selecting and wiring sensors on a breadboard, through firmware development in the Arduino IDE, to serving a local web interface and finally integrating with a cloud-hosted real-time database. The two-stage methodology (local server first, cloud integration second) proved effective for isolating and resolving issues at each layer of the IoT stack independently. The resulting system demonstrates a functional multi-sensor monitoring and relay-control platform that can be accessed both locally and remotely, forming a solid foundation for further development such as persistent data logging, mobile application integration, and additional sensor automation rules.

11. Acknowledgement

This work was carried out as part of the IoT Workshop and IoTrix Hackathon 1.0, organized by the IEEE BUBT Student Branch, with support from the event's technical and collaboration partners. The authors thank the organizing team and mentors for their guidance throughout the workshop sessions.